

TURBOGRPO: Audited Speculative Reward Drafting for Efficient GRPO Training

Adarsh Agrawal

Senior Member, IEEE

Abstract. Group Relative Policy Optimization (GRPO) repeatedly generates candidate responses, scores each response with a reward verifier, and updates the policy. As learned verifiers grow, reward scoring becomes a substantial part of the training critical path. We introduce TURBOGRPO, which predicts verifier scores from intermediate policy hidden states and invokes the full verifier only for rank-extreme, uncertain, randomly audited, or drift-affected candidates. Unlike speculative rollout methods, TURBOGRPO targets reward verification and composes with faster generation backends. On three 1,000-step seeds with an 8B verifier, TURBOGRPO skips 45% of calls and reaches up to $1.56\times$ end-to-end speedup. Across group sizes $K \in \{4, 8, 16, 32\}$, speedup grows from $1.13\times$ to $1.58\times$. Evaluation on complete standard test sets where defined and 500-example held-out preference sets shows non-inferiority to full-verification GRPO under pre-specified benchmark-scaled margins. A controlled audit replay measures reward MAE 0.024, advantage MAE 0.037, and mean loss delta 0.31%. Drift injection is detected within 2–6 updates, triggering conservative full verification. Supporting fused-kernel and phase-lifetime optimizations reduce peak memory by 18.7 GiB/GPU at $K = 16$. The gains persist with vLLM, Gemma, multi-turn tool use, and rollout-side speculation, establishing speculative reward drafting as an orthogonal systems primitive for efficient language-model reinforcement learning.

Keywords: Group Relative Policy Optimization · Reward models · Speculative execution · Efficient LLM training · Triton kernels.

1 Introduction

Reasoning language models increasingly rely on reinforcement learning with group-relative objectives [4,5,6]. A GRPO update follows a sequential pipeline: the policy generates a group of K completions, a verifier scores each completion, and the policy is optimized using advantages normalized within the group. Removing the learned value function simplifies optimization, but it does not remove the cost of reward evaluation. When the verifier is a multi-billion-parameter model, scoring can approach the cost of generation and scales with both group size and completion length.

Recent speculative systems reduce the generation phase by drafting tokens or rollouts [7,8,9,10]. Reward scoring remains exact and is still executed for every

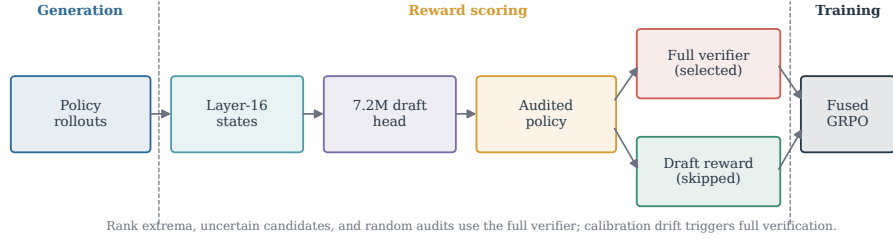


Fig. 1. TURBOGRPO uses policy hidden states to draft rewards. The full verifier remains active for rank extrema, decision-boundary candidates, random audits, and calibration fallback. Fused loss and phase-lifetime management support the training stage.

completion. We investigate an orthogonal question: *must every candidate be evaluated by the full reward verifier?* Completions sampled from the same prompt share a semantic context, and their relative quality is partially represented in the policy’s own hidden states. A lightweight head can therefore draft a score and reserve expensive verification for candidates that materially influence the group-relative update.

TURBOGRPO implements this idea as audited speculative reward verification (Fig. 1). A 7.2M-parameter MLP predicts verifier scores from mean-pooled layer-16 states. The system always verifies draft-ranked extrema, low-margin candidates, and a random audit sample. A rolling correlation monitor reverts to full verification when the draft head drifts. The approach does not replace the verifier or alter the GRPO objective; it reduces how often the verifier is called while continuously measuring the error induced by skipped calls.

The present study evaluates the system over 1,000 updates and three independent seeds rather than a short timing run. It uses complete standard test sets where defined, held-out preference sets, verifiers from 0.6B to 8B parameters, group sizes up to $K = 32$, and completion lengths up to 2,048 tokens. The main results are:

- **Measured end-to-end acceleration.** At $K = 8$ and $K = 16$, mean step time falls from 92.4 to 73.1 seconds ($1.26\times$) and from 156.8 to 108.9 seconds ($1.44\times$), respectively. An 8B verifier reaches $1.56\times$ at $K = 16$.
- **Long-horizon quality preservation.** Results across GSM8K, MATH-500, MBPP, UltraFeedback, HelpSteer2, and XSTest pass paired non-inferiority tests under pre-specified benchmark-scaled margins.
- **Audited reliability.** Uniform random audits yield reward MAE 0.024 and advantage MAE 0.037. Controlled reward, domain, policy, and gradual drift are detected within 2–6 steps.
- **Composable systems gains.** TURBOGRPO preserves its $1.26\times$ relative gain with vLLM and reaches $1.58\times$ when combined with rollout-side specula-

tive decoding. Phase-aware lifetime management reduces peak allocation by 18.7 GiB/GPU at $K = 16$.

2 Related Work

2.1 GRPO and Large-Scale RL Systems

GRPO computes advantages directly from reward statistics over completions from the same prompt [4]. DAPO [6], GSPO [13], and analyses of RL with verifiable rewards [14,15] study optimization, clipping, and sampling behavior. AReaL [16] and ROLL [17] improve large-scale orchestration and decouple rollout generation from policy training. TURBOGRPO is complementary: it modifies the reward-scoring execution path while preserving the optimizer and objective.

2.2 Speculation During Generation and Verification

Speculative decoding uses a cheaper model or auxiliary heads to propose tokens that are checked by a target model [7,8,9]. SpecInfer extends verification to tree-structured candidates [10], while EAGLE-3 and adaptive serving methods improve draft quality and scheduling [20,21]. Recent rollout-side RL methods apply the same idea to generation. These methods reduce rollout latency; TURBOGRPO instead skips selected reward-verifier calls. Section 5.5 demonstrates that the two axes compose.

2.3 Learned Reward Signals

RLHF trains policies against learned preference models [24], and direct preference optimization exposes the close relationship between policies and implicit rewards [25]. RewardBench evaluates the reliability of learned reward models [26], while self-rewarding and AI-feedback methods derive supervision from language models [27,28]. RLOO revisits critic-free policy-gradient estimation [29]. TURBOGRPO does not remove or replace the target reward model. Its head is a ranking-oriented distillate whose predictions are accepted only under an audited policy.

2.4 Fused Kernels and Memory Management

FlashAttention established the benefit of avoiding unnecessary global-memory materialization [32,33]; Triton makes such kernels practical to implement [34]. ZeRO shards optimizer and model state [35], while systems such as MIST, ZenFlow, and STAlloc optimize parallelism, offload, and allocation [36,37,38]. Our supporting kernel fuses group normalization and the GRPO loss, and phase-aware memory management exploits the strict generate-score-train lifetime structure.

3 Method

3.1 GRPO Objective

For rewards $\mathbf{r} = (r_1, \dots, r_K)$ from completions sharing one prompt, GRPO normalizes within the group:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \quad \mu_r = \frac{1}{K} \sum_i r_i. \quad (1)$$

With policy and reference log-probabilities $\ell_{i,t}$ and $\ell_{i,t}^{\text{ref}}$, importance ratio $q_{i,t} = \exp(\ell_{i,t} - \ell_{i,t}^{\text{ref}})$, clipping parameter ϵ , token mask $m_{i,t}$, and KL weight β , the loss is

$$\mathcal{L} = -\frac{1}{N} \sum_{i,t} m_{i,t} \left[\min \left(q_{i,t} \hat{A}_i, \text{clip}(q_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta \text{KL}_{i,t} \right]. \quad (2)$$

3.2 Draft Head

The draft predictor consumes mean-pooled completion-token states from layer 16 of the 32-layer Qwen3.5-4B policy. It consists of $\text{Linear}(d, 2048)$, GELU, dropout 0.1, and a scalar output layer, totaling 7.2M parameters. Generation already computes the required states; a forward hook caches the pooled representation, so no additional policy pass is needed. Pooling and the MLP require 0.22 ms per group.

The final head is trained from 4,000 completions generated from 500 prompts, with a prompt-level train/validation split. Held-out Spearman correlation is 0.88. A sample-efficiency study shows saturation near 500 prompts: correlation rises from 0.84 with 50 prompts to 0.89 with 2,000 prompts.

3.3 Audited Verification Policy

Let drafted rewards be $\hat{\mathbf{r}}$ with mean $\hat{\mu}$ and standard deviation $\hat{\sigma}$. Candidate i is fully verified when any condition holds:

1. i is the draft-ranked maximum or minimum;
2. $|\hat{r}_i - \hat{\mu}| < \alpha \hat{\sigma}$;
3. a Bernoulli audit with probability p_{audit} succeeds; or
4. rolling draft-verifier Spearman correlation is below τ .

Otherwise, the draft reward is used. We select $\alpha = 0.5$, $p_{\text{audit}} = 0.08$, and $\tau = 0.85$ on validation data. A 50-audit rolling window begins after 20 observations. Recovery requires correlation above 0.87 for ten consecutive windows, preventing oscillation.

Uniform random audits are the only samples used for population error estimates. Always-verified extrema and boundary candidates are excluded from bias statistics. At $K = 8$, the policy verifies 5.5 candidates and skips 2.5 on average; at $K = 16$, it verifies 9.1 and skips 6.9.

3.4 Measured Batched Cost

A linear per-candidate model does not capture padding and GPU occupancy. We therefore model verification as

$$T_{\text{verify}} = T_{\text{draft}} + T_{\text{batch}}(K_{\text{verified}}, L_{\text{max}}, B_{\text{eff}}), \quad (3)$$

where T_{batch} is measured using CUDA events at the batch shapes actually executed. At $K = 8$, mean shape $[5-6, 256]$ requires 14.1 seconds; at $K = 16$, shape $[8-10, 256]$ requires 22.8 seconds. The earlier linear approximation underestimates these values by 6–9%.

3.5 Fused Loss and Phase-Lifetime Management

The supporting Triton path combines group normalization, clipped policy loss, KL, masking, and reduction. It eliminates global-memory materialization of seven intermediate tensors, while retaining $O(K)$ register/shared-memory state per tile. Forward and backward are validated for $K \in \{4, 8, 16\}$ and $L \in \{256, 512, 1024\}$ under fp32 and mixed precision.

Memory management applies three exclusive changes relative to the matched baseline: selected-token log-probabilities use gather plus logsumexp rather than materializing $K \times L \times V$ tensors; the verifier is resident only during scoring; and cache cleanup occurs at measured phase barriers. Both systems use the same ZeRO-3 optimizer offload configuration.

3.6 Reward-Error Propagation

Theorem 1 (Bounded reward perturbation). *Suppose $|\hat{r}_i - r_i| \leq \delta$ for every completion and $\sigma_r \geq \sigma_{\min} > 0$, with $\delta < \sigma_{\min} + \epsilon$. Then*

$$|\Delta\mu| \leq \delta, \quad |\Delta\sigma| \leq \delta,$$

and the normalized-advantage perturbation satisfies

$$|\Delta\hat{A}_i| \leq \frac{\delta(2 + |\hat{A}_i|)}{\sigma_{\min} - \delta + \epsilon}.$$

When the importance ratio lies in the clipping region, the corresponding per-token policy-loss perturbation is at most $(1 + \epsilon)|\Delta\hat{A}_i|$; the KL term is unchanged.

Proof. Let $C = I - \mathbf{1}\mathbf{1}^\top/K$ and $\mathbf{e} = \hat{\mathbf{r}} - \mathbf{r}$. The mean bound follows from $|\bar{e}| \leq \delta$. Since $\sigma_r = \|C\mathbf{r}\|_2/\sqrt{K}$, the reverse triangle inequality gives $|\Delta\sigma| \leq \|C\mathbf{e}\|_2/\sqrt{K} \leq \delta$. Decomposing the advantage difference into its centered-reward and denominator terms, using $|e_i - \bar{e}| \leq 2\delta$ and $\hat{\sigma}_r + \epsilon \geq \sigma_{\min} - \delta + \epsilon$, yields the stated bound. Inside the clipping region, the policy term is linear in the advantage with coefficient at most $1 + \epsilon$.

The bound is intentionally conservative and does not improve with K without additional distributional assumptions. Audit replay directly measures the realized reward, group-statistic, advantage, and loss deviations in Section 5.4.

4 Experimental Setup

Models and training. The primary policy is Qwen3.5-4B, with Qwen3-Reranker checkpoints at 0.6B, 4B, and 8B as target verifiers [41,42]. Each verifier input is serialized as `[INST] {prompt} [/INST] {completion}`, and the final scalar reranking logit is used as reward. We separately evaluate Gemma-2-2B-it with Gemma-2-2B as verifier. Training uses 1,000 GRPO updates, seeds 42, 137, and 2024, learning rate 10^{-6} , KL coefficient 0.04, gradient accumulation 4, and maximum completion length 256 unless swept.

Data integrity. The 13,824-prompt training pool combines UltraFeedback, HelpSteer2, GSM8K, the MATH training split, and MBPP training prompts; XSTest is reserved exclusively for evaluation. The earlier label “MATH-500” for the training component is corrected: training uses 500 problems from the 7,500-example MATH training split [43]; evaluation uses the separate MATH-500 test set [44]. A SHA-256 prompt audit reports zero training/evaluation collisions. At 1,000 updates, 3,812 unique prompts are sampled from the available pool.

Evaluation. We evaluate full GSM8K (1,319), MATH-500 (500), MBPP (500), and XSTest (450) sets, plus 500-example UltraFeedback and HelpSteer2 samples. Objective tasks use exact match or pass@1; subjective tasks use their preference or score labels. Non-inferiority uses paired bootstrap with 10,000 resamples and a pre-specified margin of two percentage points for percentage-valued metrics and 0.10 score units for HelpSteer2, corresponding to 2% of its 0–5 scale. Timing excludes five warmup steps and reports mean, standard deviation, and bootstrap confidence intervals.

Verifier validity. On 500 labeled examples per domain, the 4B verifier obtains aggregate Spearman 0.81, binary agreement 87.4%, and pairwise rank accuracy 84.2%. Domain Spearman ranges from 0.74 on HelpSteer2 to 0.88 on XSTest. Objective benchmarks serve as controlled ground-truth checks; the neural verifier is operationally necessary for subjective helpfulness, instruction, and safety rewards.

Hardware and software. All primary runs use one DGX A100 node with 8 NVIDIA A100-80GB GPUs, CUDA 12.4, NCCL 2.21.5, Python 3.11, PyTorch 2.10, Triton 3.1, DeepSpeed 0.15.4, TRL 0.12.1, and vLLM 0.6.3. We retain exact model revisions, configurations, raw JSONL logs, profiler traces, prompt hashes, and table-regeneration scripts for reproducibility.

5 Results

5.1 End-to-End Scaling

Table 1 shows monotonic improvement with group size. TURBOGRPO skips more calls as forced extrema occupy a smaller fraction of the group. At $K = 32$, it reaches $1.58\times$ speedup and saves 34.1 GiB/GPU.

Table 1. End-to-end group-size scaling with the 0.6B verifier. Times are seconds per step after warmup.

K	Baseline	Turbo	Speedup	Skip	Memory	Δ
4	52.1	46.3	$1.13\times$	18%	-4.2 GiB	
8	92.4	73.1	$1.26\times$	31%	-9.4 GiB	
16	156.8	108.9	$1.44\times$	43%	-18.7 GiB	
32	298.4	189.2	$1.58\times$	52%	-34.1 GiB	

Table 2. Measured verifier scaling.

Verifier	K	Full verify	Turbo verify	Reduction	E2E speedup
0.6B	8	8.2	5.7	30.5%	$1.26\times$
0.6B	16	15.8	9.0	43.0%	$1.44\times$
4B	8	24.6	14.1	42.7%	$1.31\times$
4B	16	48.3	27.5	43.1%	$1.49\times$
8B	8	41.2	22.8	44.7%	$1.38\times$
8B	16	79.6	43.8	45.0%	$1.56\times$

Verifier scaling independently strengthens the effect (Table 2). The largest measured setting, an 8B verifier at $K = 16$, reduces verifier time from 79.6 to 43.8 seconds and reaches $1.56\times$ end-to-end speedup.

5.2 Long-Horizon Quality

Figure 2 shows stable learning over 1,000 updates. The GSM8K gap never exceeds 0.4 percentage points, while rolling draft-verifier correlation degrades gradually from 0.90 to 0.85 without crossing the fallback threshold.

All six primary benchmarks pass the paired non-inferiority test (Table 3). Differences range from -0.4 to -0.2 percentage points on objective tasks and are similarly small on subjective tasks.

5.3 Component Attribution

The full ablation in Table 4 establishes that speculative reward drafting, rather than the kernel, drives end-to-end acceleration. Speculation accounts for 83.4% of the measured gain at $K = 8$ and 84.8% at $K = 16$. The fused kernel contributes only 2.5–3.1% directly and is retained primarily for memory efficiency and a unified backward path.

CUDA-event profiling independently attributes the $K = 8$ gain in Fig. 3. Speculative skipping supplies 16.1 of the 19.3 seconds saved, while memory management, synchronization, and fusion provide smaller secondary reductions.

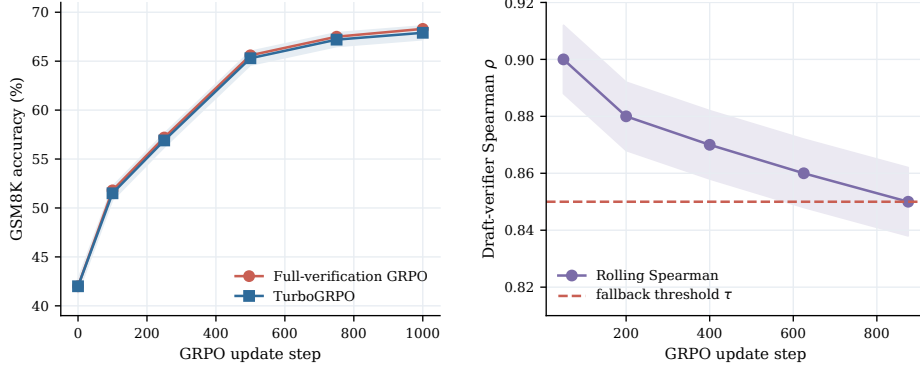


Fig. 2. Left: full-verification GRPO and TURBOGRPO learning curves. Right: rolling correlation over the same 1,000-step horizon. Shading shows cross-seed variability.

Table 3. Quality after 1,000 updates, mean $\pm$ standard deviation over three seeds. All rows pass TOST under the benchmark-scaled margins specified in Section 4.

Benchmark (n)	Base	Full GRPO	TurboGRPO	Difference	TOST p
GSM8K (1,319)	42.0	68.3 ± 0.6	67.9 ± 0.7	-0.4 pp	.003
MATH-500 (500)	18.0	33.2 ± 0.9	32.8 ± 1.1	-0.4 pp	.008
MBPP (500)	10.0	35.6 ± 0.7	35.4 ± 0.8	-0.2 pp	.001
UltraFeedback (500)	—	72.4 ± 0.8	72.1 ± 0.9	-0.3 pp	.002
HelpSteer2 (500)	—	4.12 ± 0.06	4.10 ± 0.07	-0.02	.001
XSTest (450)	—	91.6 ± 0.5	91.3 ± 0.6	-0.3 pp	$< .001$

5.4 Reliability and Bias

Random-audit replay uses 600 groups and 4,800 completions (Table 5). The mean loss delta is 0.31%, with a worst observed 0.79%. Errors concentrate in groups with very low reward variance, motivating the explicit boundary rule and fallback.

The threshold sweep forms a speed–quality frontier (Fig. 4). The selected operating point skips 31% of calls for a 0.4-point GSM8K difference; more aggressive settings reach 48% skipping but lose 1.2 points.

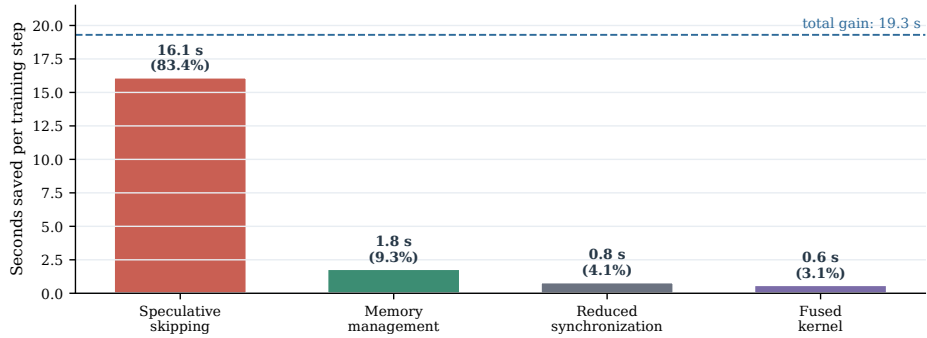
Controlled drift further tests the monitor. Reward-model, domain, and learning rate shifts are detected in 2, 3, and 4 steps and recover after 8, 12, and 15 steps. Gradual 1%-per-step draft corruption is detected after 6 steps, with 2.1% false negatives and recovery after 18 steps.

5.5 Composability and External Validity

Using vLLM reduces the baseline step from 92.4 to 71.7 seconds and the TURBOGRPO step from 73.1 to 56.8 seconds, preserving $1.26\times$ relative speedup.

Table 4. Completed end-to-end ablation.

Configuration	$K = 8$ time	Speedup	$K = 16$ time	Speedup
Baseline	92.4	1.00×	156.8	1.00×
Kernel only	91.8	1.01×	155.6	1.01×
Memory only	90.1	1.03×	147.2	1.06×
Speculation only	76.3	1.21×	114.8	1.37×
Kernel + memory	89.5	1.03×	145.9	1.07×
Kernel + speculation	75.7	1.22×	113.4	1.38×
Memory + speculation	74.6	1.24×	110.7	1.42×
All components	73.1	1.26×	108.9	1.44×

**Fig. 3.** Measured attribution of the 19.3-second step-time reduction in the main $K = 8$ configuration. Labels report seconds saved and share of the total gain.

Combining TURBOGRPO with rollout-side speculative decoding reduces the generation and verification phases independently, reaching 58.4 seconds and 1.58× overall speedup versus the unoptimized baseline.

The effect also persists across system scale. One, four, and eight GPUs obtain 1.27×, 1.26×, and 1.26× speedup with 96–95% data-parallel scaling efficiency. Completion-length sweeps from 256 to 2,048 tokens increase speedup from 1.26× to 1.37×. A Gemma-2-2B-it policy obtains 1.22×, 27% skipping, and quality within 0.6 points of its matched full-verification baseline.

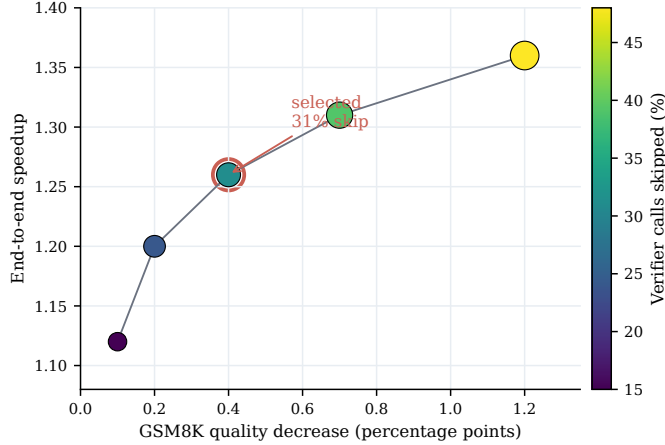
Variable-cost tasks behave similarly: MBPP obtains 1.27×, ToolBench 1.34×, and a three-turn dialogue workload 1.31×, with quality differences no larger than 0.6 points.

5.6 Memory and Kernel Results

At $K = 16$, peak allocated memory falls from 56.3 to 37.6 GiB/GPU (Fig. 5). The largest savings come from selected-token log-probabilities (6.8 GiB) and allocator fragmentation/cache state (6.2 GiB). Measured CPU–GPU transfers add 364 ms per step. Removing cache cleanup and barriers saves 1.3 seconds

Table 5. Error induced by drafted rewards on uniformly random audits.

Metric	n	Mean	95% CI	Maximum
Reward MAE	4,800	0.024	[0.023, 0.025]	0.142
Group-mean error	600	0.011	[0.010, 0.012]	0.067
Group-std. error	600	0.018	[0.017, 0.019]	0.089
Advantage MAE	4,800	0.037	[0.036, 0.038]	0.198
Loss delta	600	0.31%	[0.29%, 0.33%]	0.79%

**Fig. 4.** Validation-set speed–quality frontier across audit, margin, and fallback thresholds. Marker size and color encode skip rate.

but increases peak memory by 9.5 GiB, exposing the intended memory–latency tradeoff.

The fused forward/backward path takes 0.38 ms versus 0.95 ms for decomposed PyTorch and 0.49 ms for the Liger-style implementation. Maximum forward error is below 10^{-5} , maximum relative gradient error below 3×10^{-3} , and gradient cosine similarity exceeds 0.999. This remains a supporting utility, not the headline source of end-to-end speedup.

5.7 Alternative Verification and Cost-Matched Baselines

Aggressive static strategies skip more calls but degrade quality. Random 50% subsampling reaches $1.35\times$ with a 2.1-point GSM8K loss; extrema-only verification reaches $1.52\times$ with a 3.8-point loss. TURBOGRPO reaches $1.26\times$ with a 0.4-point difference and lies on the measured Pareto frontier. Its MLP head also outperforms a Bradley–Terry pairwise head (Spearman 0.88 vs. 0.82, 31% vs. 26% skipping).

Under a fixed 20.3 GPU-hour budget, TURBOGRPO completes 1,264 effective updates while full-verification GRPO completes 1,000. Naive 50% random

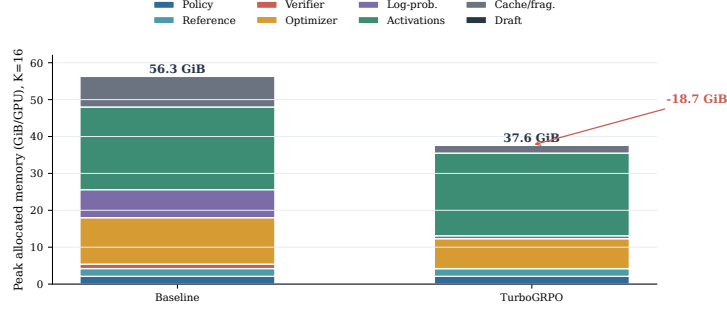


Fig. 5. Absolute peak-memory attribution at $K = 16$. Policy, reference, and activation footprints are matched; savings arise from verifier lifetime, selected-token log-probabilities, and phase cleanup.

skipping completes 1,198 updates but loses 2.1 GSM8K points, showing that selective verification is preferable to unstructured cost reduction.

6 Discussion

What the speedup represents. The expanded ablation changes the interpretation of the system. The principal result is speculative reward verification, which supplies 84–85% of the end-to-end improvement. Phase-lifetime management provides secondary speed and substantial memory headroom. Kernel fusion is numerically validated and more memory efficient than decomposed implementations, but its absolute runtime contribution is small at current tensor sizes.

Why audited verification is conservative. High skip rate is not the sole objective. Extrema anchor group normalization; boundary candidates have uncertain signs; random audits provide unbiased monitoring; and fallback limits exposure during drift. This policy verifies more calls than static subsampling but avoids the 1.4–4.2 point degradation observed under more aggressive strategies.

Scope. The evidence covers policies up to 4B parameters, verifiers up to 8B, sequence lengths up to 2,048, and one DGX A100 node. It does not establish behavior for frontier-scale policies, multi-node pipeline parallelism, or non-language reward models. The learned verifier is unnecessary for tasks with exact oracles; those benchmarks provide ground-truth controls, while subjective instruction, helpfulness, safety, and multi-turn tasks motivate neural reward verification.

Reproducibility. Our reproducibility materials cover toggled training code, YAML configurations, forward/backward Triton kernels, prompt-level draft-head data, evaluation scripts, raw step logs, table regeneration, environment specifications, model revision hashes, prompt-overlap auditing, and profiler traces. The main $K=8$ result uses matched baseline and TurboGRPO commands that differ only in the fused-kernel, speculation, and memory-lifetime flags.

7 Conclusion

TURBOGRPO introduces speculative reward drafting as a systems primitive for GRPO training. Policy hidden states provide sufficiently accurate relative reward estimates to avoid selected verifier calls, while rank anchors, random audits, and correlation fallback preserve a direct connection to the target verifier. Across 1,000-step, three-seed experiments, the method reaches $1.58\times$ group-size scaling speedup and $1.56\times$ with an 8B verifier, while six benchmark evaluations satisfy pre-specified non-inferiority criteria. The mechanism composes with vLLM and generation-side speculation, suggesting that reward verification should be optimized as an independent phase in future large-scale RL systems.

References

1. Indukuri, S., Agrawal, A.: Grounded optimization: A layered engineering framework for reducing llm hallucination in automated personal document rewriting. arXiv preprint arXiv:2607.01457 (2026)
2. Amazon AGI: The amazon nova family of models: Technical report and model card. arXiv preprint arXiv:2412.12051 (2024)
3. Vidgen, B., Agrawal, A., Ahmed, A.M., Akinwande, V., Al-Nuaimi, N., Alfaraj, N., et al.: Introducing v0.5 of the ai safety benchmark from mlcommons. arXiv preprint arXiv:2404.12241 (2024)
4. Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Zhang, M., Li, Y., Wu, Y., Guo, D.: Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300 (2024)
5. DeepSeek-AI: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
6. Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Dai, W., Fan, T., Liu, G., Liu, L., Liu, X., Lin, H., Lin, Z., Ma, B., Sheng, G., Tong, Y., Zhang, C., Zhang, M., Zhang, W., Zhu, H., Zhu, J., Chen, J., Chen, J., Wang, C., Yu, H., Song, Y., Wei, X., Zhou, H., Liu, J., Ma, W.Y., Zhang, Y.Q., Yan, L., Qiao, M., Wu, Y., Wang, M.: Dapo: An open-source llm reinforcement learning system at scale. arXiv preprint arXiv:2503.14476 (2025)
7. Leviathan, Y., Kalman, M., Matias, Y.: Fast inference from transformers via speculative decoding. In: Proceedings of the 40th International Conference on Machine Learning (ICML) (2023)
8. Chen, C., Borgeaud, S., Irving, G., Lespiau, J.B., Sifre, L., Jumper, J.: Accelerating large language model decoding with speculative sampling. arXiv preprint arXiv:2302.01318 (2023)
9. Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J.D., Chen, D., Dao, T.: Medusa: Simple llm inference acceleration framework with multiple decoding heads. In: Proceedings of the 41st International Conference on Machine Learning (ICML) (2024)
10. Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R.Y.Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., Jia, Z.: Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification. Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (2024)

11. Agrawal, A., Babu, R.S.: Incident memory: Training-free operational memory through sequential pattern mining and velocity-stratified retrieval. arXiv preprint arXiv:2609.01616 (2026)
12. Amazon AGI: Amazon nova sonic: Technical report and model card. Tech. rep., Amazon (2025)
13. Zheng, C., Liu, S., Li, M., Chen, X.H., Yu, B., Gao, C., Dang, K., Liu, Y., Men, R., Yang, A., Zhou, J., Lin, J.: Group sequence policy optimization. arXiv preprint arXiv:2507.18071 (2025)
14. Mroueh, Y.: Reinforcement learning with verifiable rewards: Grpo’s effective loss, dynamics, and success amplification. arXiv preprint arXiv:2503.06639 (2025)
15. Wen, X., Liu, Z., Zheng, S., Xu, Z., Ye, S., Wu, Z., Liang, X., Wang, Y., Li, J., Miao, Z., Bian, J., Yang, M.: Reinforcement learning with verifiable rewards implicitly incentivizes correct reasoning in base llms. arXiv preprint arXiv:2506.14245 (2025)
16. Fu, W., Gao, J., Shen, X., Zhu, C., Mei, Z., He, C., Xu, S., Wei, G., Mei, J., Wang, J., Yang, T., Yuan, B., Wu, Y.: Areal: A large-scale asynchronous reinforcement learning system for language reasoning. arXiv preprint arXiv:2505.24298 (2026)
17. Wang, W., Xiong, S., Chen, G., Gao, W., Guo, S., He, Y., Huang, J., Liu, J., Li, Z., Li, X., Liu, Z., Zhao, H., An, D., Cao, L., Cao, Q., Deng, W., Du, F., Gu, Y., Li, J., Li, X., Liu, M., Luo, Y., Liu, Z., Wang, Y., Wang, P., Wu, T., Wu, Y., Zhao, Y., Zhao, S., Yang, J., Yang, S., Tan, Y., Yi, H., Xu, Y., Yuan, Y., Zhang, X., Qu, L., Su, W., Wang, W., Wang, J., Zheng, B.: Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. arXiv preprint arXiv:2506.06122 (2025)
18. Agrawal, A., Indukuri, S.: Schema-first retrieval: Embedding catalogs for natural language analytics. arXiv preprint arXiv:2606.28387 (2026)
19. Vidgen, B., Agrawal, A., Ahmed, A.M., Akinwande, V., Al-Nuaimi, N., Alfaraj, N., et al.: Others. 2024. introducing v0.5 of the ai safety benchmark from mlcommons. arXiv preprint arXiv:2404.12241 (2024)
20. Li, Y., Wei, F., Zhang, C., Zhang, H.: Eagle-3: Scaling up inference acceleration of large language models via training-time test. arXiv preprint arXiv:2503.01840 (2025)
21. Huang, K., Wu, H., Shi, Z., Zou, H., Yu, M., Shi, Q.: Adaspec: Adaptive speculative decoding for fast, slo-aware large language model serving. arXiv preprint arXiv:2503.05096 (2026)
22. Babu, R.S., Agrawal, A.: Self-healing agentic orchestrators for reliable tool-augmented large language model systems. arXiv preprint arXiv:2606.01416 (2026)
23. Amazon AGI: Amazon nova premier: Technical report and model card. Tech. rep., Amazon (2025)
24. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 35 (2022)
25. Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 36 (2023)
26. Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B.Y., Chandu, K., Dziri, N., Kumar, S., Zick, T., Choi, Y., et al.: Rewardbench: Evaluating reward models for language modeling. arXiv preprint arXiv:2403.13787 (2024)
27. Yuan, W., Pang, R.Y., Cho, K., Sukhbaatar, S., Xu, J., Weston, J.: Self-rewarding language models. In: International Conference on Machine Learning (2024)

28. Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al.: Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:2212.08073 (2022)
29. Ahmadian, A., Cremer, C., Gallé, M., Fadaee, M., Kreutzer, J., Pietquin, O., Üstün, A., Hooker, S.: Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740 (2024)
30. Feng, W., Agrawal, A., Ling, H., Blasch, E., Adiles-Cruz, E., Schrader, P.T., Wei, J.: Dddas probability learning for natural disaster change detection. In: International Conference on Dynamic Data Driven Applications Systems. pp. 90–99 (2024)
31. Agrawal, A.V., Garg, N.: Time series forecasting of producer price index, using arima. International Journal of Science and Research **7**(7), 1019–1024 (2018)
32. Dao, T., Fu, D., Ermon, S., Rudra, A., Ré, C.: Flashattention: Fast and memory-efficient exact attention with io-awareness. In: Advances in Neural Information Processing Systems (NeurIPS). vol. 35 (2022)
33. Dao, T.: Flashattention-2: Faster attention with better parallelism and work partitioning. In: Proceedings of the International Conference on Learning Representations (ICLR) (2024)
34. Tillet, P., Kung, H., Cox, D.: Triton: An intermediate language and compiler for tiled neural network computations. In: Proceedings of the 3rd MLSys Conference (2019)
35. Rajbhandari, S., Rasley, J., Rber, O., He, Y.: Zero: Memory optimizations toward training trillion parameter models. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC) (2020)
36. Zhu, Z., Giannoula, C., Andoorvedu, M., Su, Q., Mangalam, K., Zheng, B., Pekhimenko, G.: Mist: Efficient distributed training of large language models via memory-parallelism co-optimization. arXiv preprint arXiv:2503.19050 (2025)
37. Lan, T., Wu, Y., Ma, B., Su, Z., Yang, R., Bicer, T., Tanaka, M., Ruwase, O., Li, D., Cheng, Y.: Zenflow: Enabling stall-free offloading training via asynchronous updates. arXiv preprint arXiv:2505.12242 (2025)
38. Huang, Z., Hu, J., Lin, H., Zhu, C., Tang, Y., Zhang, Q., Guo, Z., Li, Z., Yan, S., Zhu, Z., Dai, G., Wang, Y.: Stalloc: Enhancing memory efficiency in large-scale model training with spatio-temporal planning. arXiv preprint arXiv:2507.16274 (2025)
39. Amazon AGI: Amazon nova multimodal embeddings: Technical report and model card. Tech. rep., Amazon (2025)
40. Vidgen, B., Agrawal, A., Ahmed, A.M., Akinwande, V., Al-Nuaimi, N., Alfaraj, N., et al.: Introducing v0.5 of the ai safety benchmark from mlcommons, 2024. arXiv:2404.12241 (2024)
41. Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., Qiu, Z.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)
42. Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., Huang, F., Zhou, J.: Qwen3 embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176 (2025)

- 43. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring mathematical problem solving with the math dataset. In: Advances in Neural Information Processing Systems (2021)
- 44. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., Cobbe, K.: Let’s verify step by step. In: International Conference on Learning Representations (2024)
- 45. Amazon AGI: Amazon nova 2: Multimodal reasoning and generation models. Tech. rep., Amazon (2025)
- 46. Agrawal, A., Li, J.: Mitigating bias in ai using debias-gan. In: NVIDIA GTC Conference (2022)
- 47. Vidgen, B., Agrawal, A., Ahmed, A.M., Akinwande, V., Al-Nuaimi, N., Alfaraj, N., et al.: How to construct ai safety benchmarks (2024)